
逢甲大學 軟體框架設計 114-2 期末專題

公司車輛管理系統

Corporate Vehicle Management System

以 OOD 原則與 10 個 GoF 設計模式為核心的全棧實作

組員

王建葦 D1210799 • 陳稚翔 D1249623

demo.jw-albert.dev

問題與動機

企業車輛多以紙本與 Excel 借用，缺乏狀態追蹤與審核軌跡。

目標 — 以物件導向設計打造可追蹤、可審核、且能持續擴充的車輛借用平台。

01 時段衝突

同一輛車重複借用，事後才發現。

02 審核不透明

申請、核准、出車、還車缺乏完整紀錄。

03 難以擴充

通知方式、衝突規則寫死，改一處動全身。

系統概觀與借車工作流程



- 使用者認證 · JWT
- 車輛管理
- 借車申請
- 審核 workflow
- 保養管理
- 違規管理
- 通知收件夾
- 稽核日誌

技術選型

後端	前端	建構與部署
Spring Boot3.3.4	Vue3.5	Maven Wrapper
Java21	TypeScript5.5	GitHub Actions · CI/CD
Spring Security · JWT0.12.6	Vite5.4	PM2 行程管理
Spring Data JPA · Flyway—	Pinia · 狀態管理2.2	Cloudflare Tunnel
Apache POI · Excel 匯出5.3.0	Vue Router · Axios4.4 / 1.7	tag 觸發自動部署至 VPS
springdoc-openapi · Swagger UI2.6.0	Chart.js · 儀表板4.5	
PostgreSQL / H2prod / dev		

分層架構

Presentation	Vue 3 SPA · REST Controllers · DTOs
Service	Borrowing · Vehicle · Maintenance · User Service
Domain	BorrowingRequest · Role · Observer · Strategy
Repository ↗	IVehicleRepository · IBorrowingRepository ... (DIP 邊界)
Infrastructure	JPA Adapters · JWT Security · Flyway Migration

依賴反轉

上層只依賴 Repository 介面；Infrastructure 在編譯期被反轉，成為實作細節。

Service 與 Domain 因此能脫離資料庫獨立測試。

設計模式總覽

10 patterns · 3 categories

BEHAVIORAL · 6

State	借車申請 5 種狀態流轉
Observer	狀態變更時的事件通知
Strategy	可替換的時段衝突演算法
Template Method	Service 層權限守衛
Chain of Responsibility	申請送審的多步驗證鏈
Command	狀態操作封裝與稽核

CREATIONAL · 2

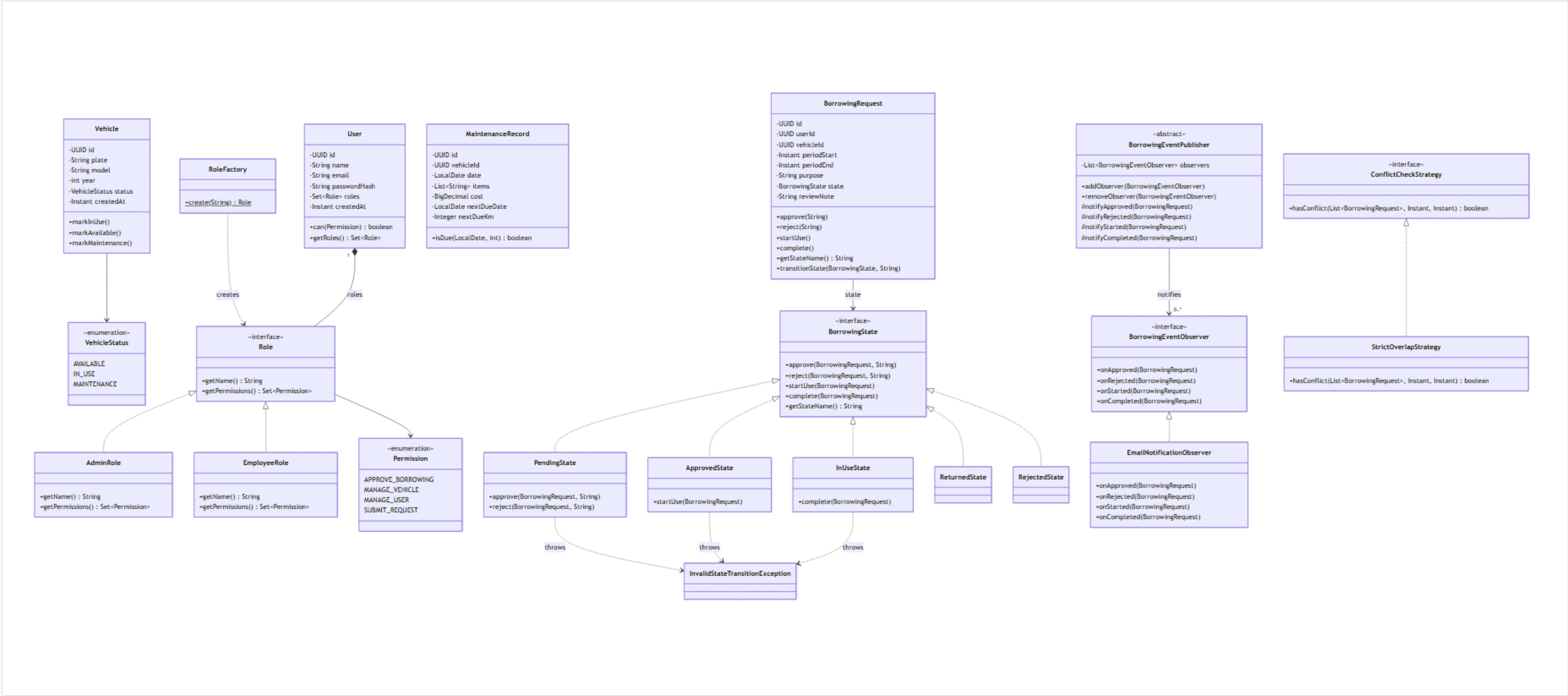
Factory Method	依角色名建立 Role 物件
Builder	從資料庫無副作用還原申請

STRUCTURAL · 2

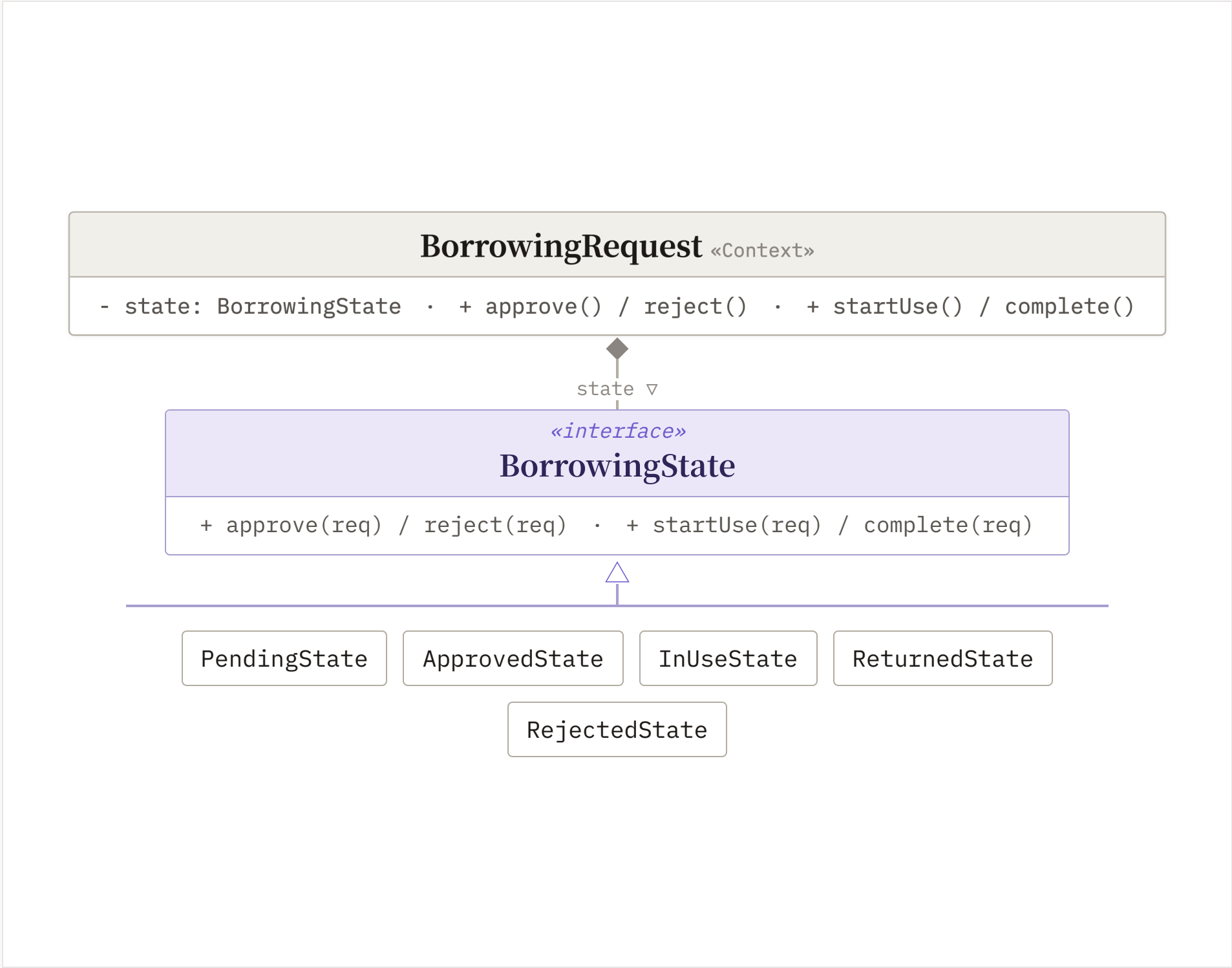
Adapter	Domain 介面橋接 JPA
Decorator	疊加時段緩衝規則

領域模型全貌

State · Factory · Observer · Strategy
在同一張類別圖中交會



State Pattern — 借車申請生命週期



為何使用

借車申請有 5 種狀態、每種只允許特定轉換；用 `if-else` 判斷狀態字串會讓 Service 充滿分支邏輯。

設計特色

- › 轉換邏輯封裝在各 `ConcreteState`，Context 僅委派。
- › 非法轉換拋 `InvalidStateTransitionException` → HTTP 422。
- › 新增狀態只加一個類別，不改 Context。

Observer Pattern — 借車事件通知



為何使用

狀態變更需通知相關人員，但通知方式（Email / 簡訊 / 推播）不該寫死在 Service 內。

設計特色

- › Service 繼承 Publisher，只發事件、不感知通知實作。
- › Spring 自動注入所有 Observer Bean（目前 Email + 站內收件夾）。
- › 新增管道 = 加一個 @Component，零修改（OCP）。

Strategy Pattern — 時段衝突檢查



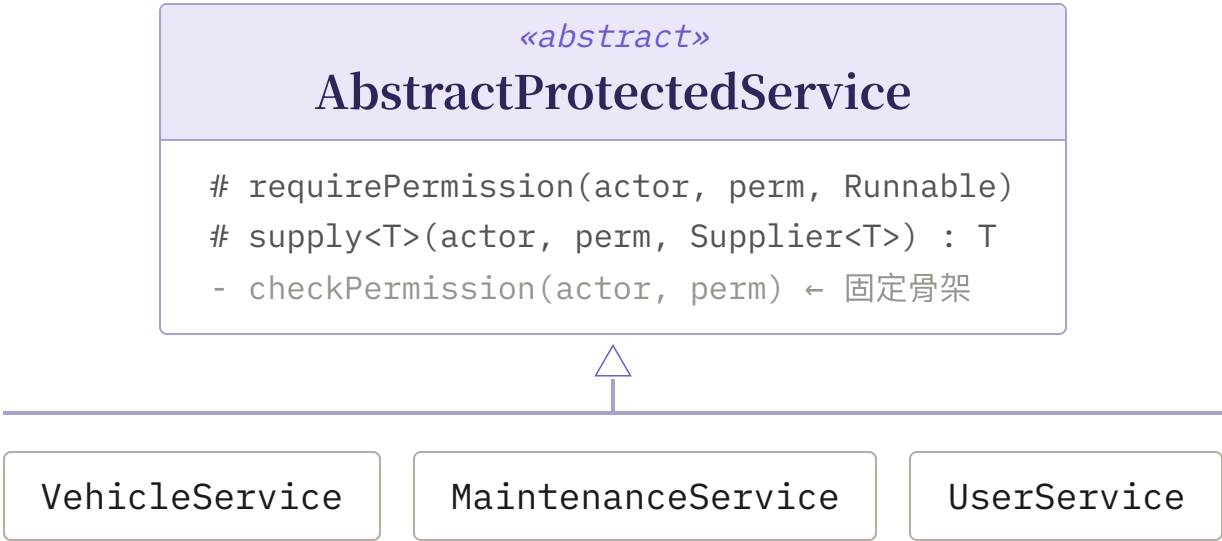
為何使用

衝突判斷演算法多樣（嚴格重疊 / 寬鬆同日 / 加緩衝）；寫死在 Service，更換邏輯就得改 Service 本身。

設計特色

- › 演算法抽成介面，由 Spring DI 注入。
- › 替換策略只需更換 @Bean 設定（OCP）。
- › 已用 Decorator 疊加緩衝時間，不改原策略。

Template Method — 服務層權限守衛



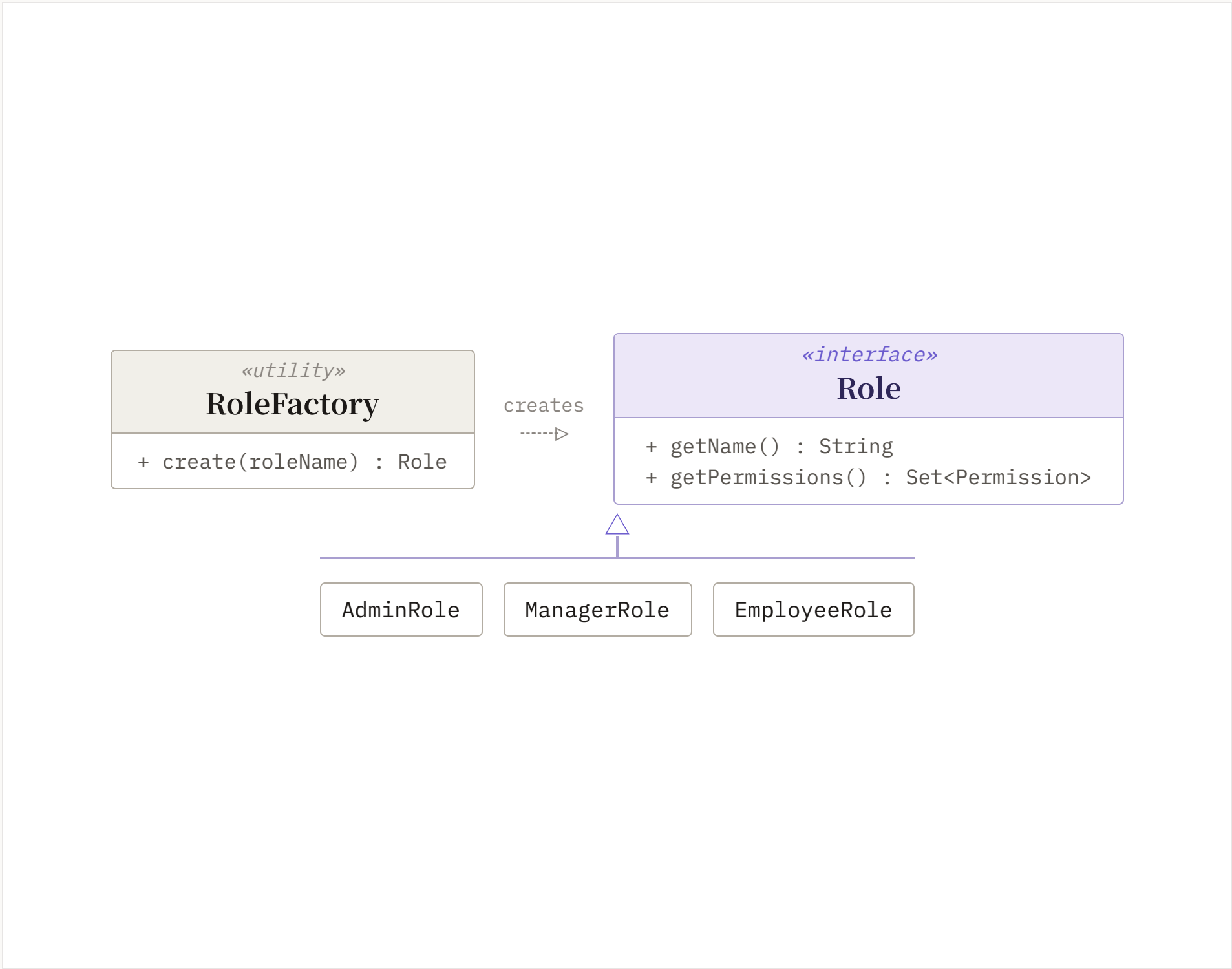
為何使用

各 Service 的 public 方法都以相同的 `if (!actor.can(...)) throw` 開頭，違反 DRY。

設計特色

- › 「先驗證、後執行」骨架固定在抽象基類。
- › 子類以 `Runnable` / `Supplier` lambda 提供業務邏輯。
- › 權限與未來稽核邏輯集中於一處。

Factory Method — 角色建立



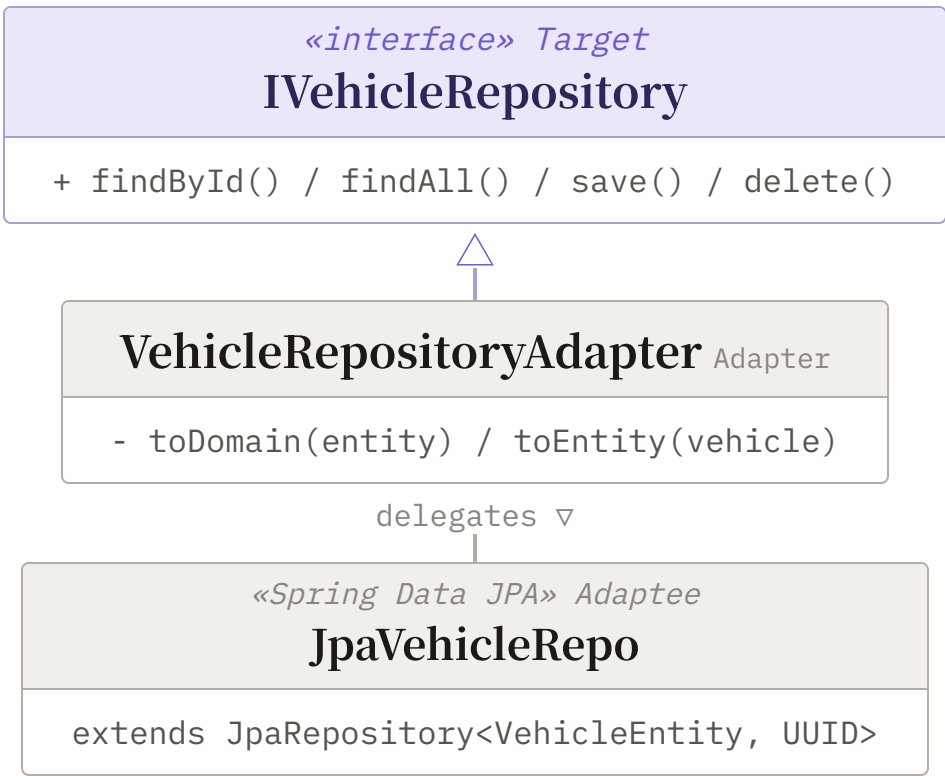
為何使用

需依字串建立對應 Role 物件；直接 new 具體類別會產生對具體型別的依賴。

設計特色

- › `RoleFactory.create()` 集中物件建立。
- › 呼叫端只依賴 Role 介面（DIP）。
- › 新增角色＝一類別＋一行 case（OCP）。

Adapter — Repository 橋接



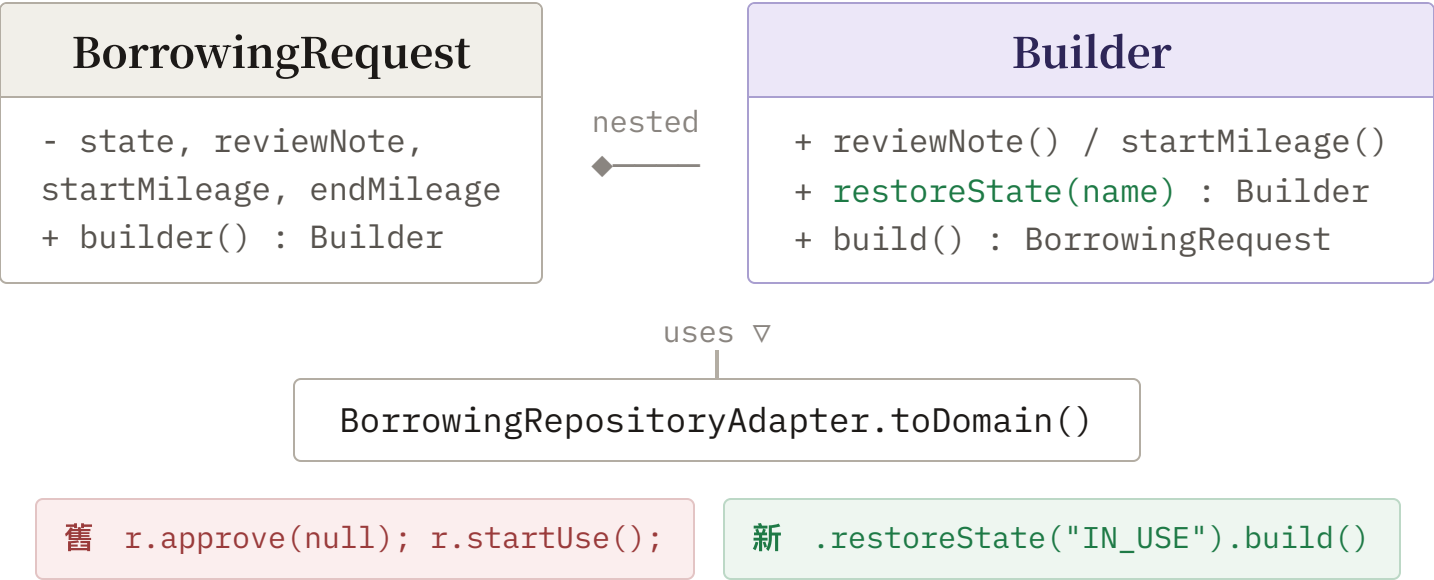
為何使用

Domain 定義 `IVehicleRepository`，但持久化用 Spring Data JPA，兩者介面不相容。

設計特色

- › Adapter 實作 Domain 介面、內部委派 JPA。
- › `toDomain()` / `toEntity()` 雙向轉換（Object Adapter）。
- › Service 不感知 JPA（DIP）；5 個 Adapter 同構。

Builder — 無副作用還原借車申請



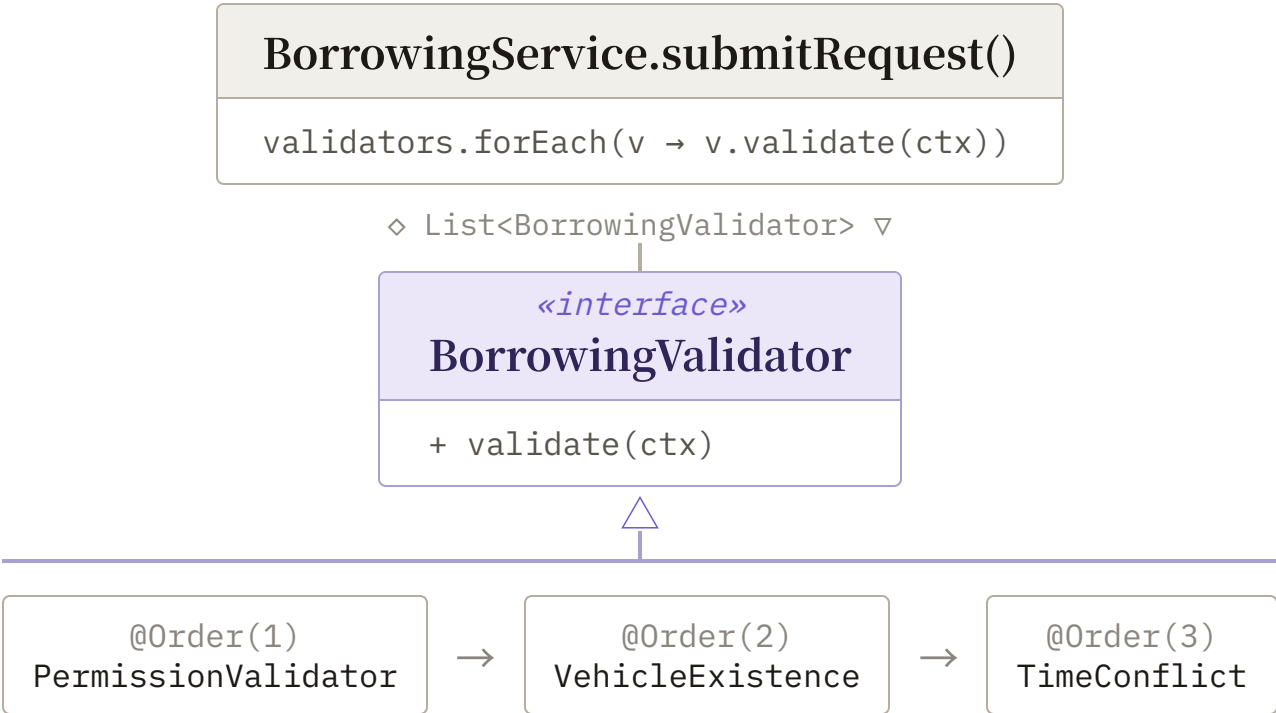
為何使用

`toDomain()` 從 DB 還原時需重播 `approve()/startUse()` 等副作用方法，會觸發 **Observer** 且邏輯脆弱。

設計特色

- › `restoreState()` 直接設定目標 State 物件。
- › 繞過 `transition` 副作用，還原安全且清晰。
- › 以模式重構 Adapter，不再重播副作用。

Chain of Responsibility — 多步驗證鏈



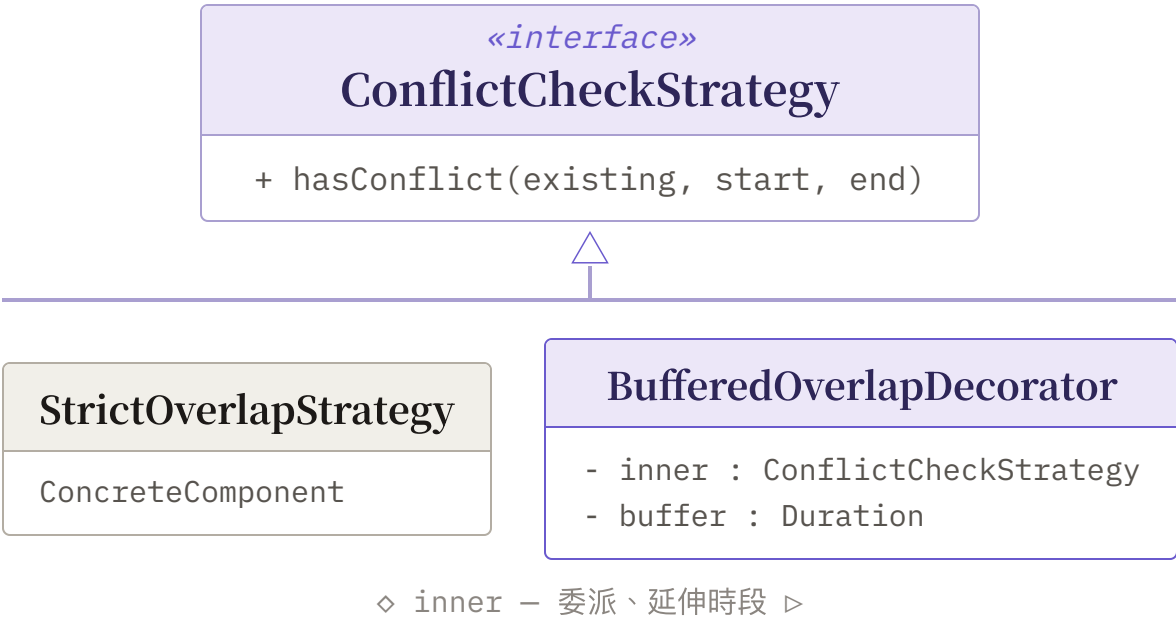
為何使用

「權限 → 車輛存在 → 時段衝突」三步驗證全寫在一個方法，新增規則需修改 Service（違反 OCP）。

設計特色

- › 各驗證步驟 `@Component @Order`，獨立可排序。
- › Service 只 `forEach` 逐一執行。
- › 新增驗證器零修改 Service（OCP）。

Decorator — 可疊加的衝突緩衝



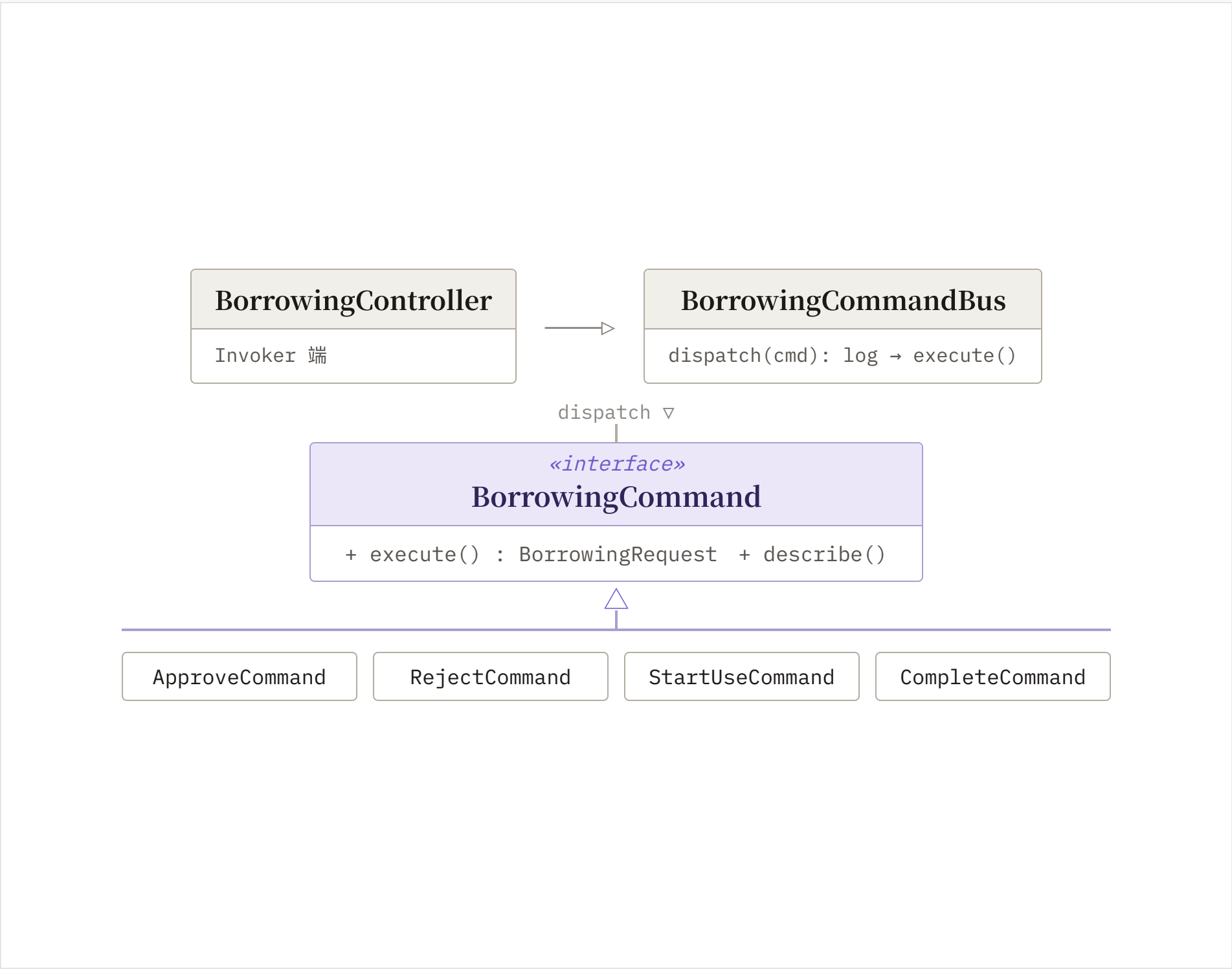
為何使用

想加「前後保留 30 分鐘緩衝」，直接改 `StrictOverlapStrategy` 會讓邏輯複雜、難以切換啟用。

設計特色

- › Decorator 包裝任意 Strategy，介面相同。
- › 委派前後延伸時段，不改被包裝策略。
- › `@Bean @Primary` 可選擇性啟用。

Command — 借車狀態操作稽核



為何使用

Controller 直接呼叫 Service，稽核、非同步等橫切關注點分散在各 endpoint，無法統一管理。

設計特色

- › 每個狀態操作封裝為一個 Command。
- › CommandBus 統一 dispatch + 記稽核日誌。
- › 未來加佇列 / 重試 / 非同步不改 Command (OCP)。

借車工作流程（依生命週期拆分）

四個 REST 端點 · 各自聚焦涉及的設計模式
PENDING → APPROVED → IN_USE → RETURNED

① 送出申請

→ PENDING

POST /api/borrowings

ChainState

員工 · Controller · Service · 責任鏈 · Repo

員工 ▶ Controller

Controller ▶ Service :
submitRequest()

Service ▶ 責任鏈 : forEach validate

1 權限 → 2 車輛存在 → 3 時段衝突

Service ▶ Repo : save · PendingState

◀ 201 Created · PENDING

② 審核

→ APPROVED

POST /{id}/approve

CommandStateObserver

審核者 · Controller · CommandBus · Service · State · Observer

審核者 ▶ CommandBus :
dispatch(ApproveCommand)

CommandBus : [AUDIT] → execute()

opt 主管 : 同部門範圍檢查

Service ▶ State : approve() → Approved

Service ▶ Observer : notifyApproved · Email

◀ 200 OK · APPROVED

拒絕對稱 → RejectedState

③ 出車

→ IN_USE

POST /{id}/start

CommandStateObserver

管理端 · CommandBus · Service · Vehicle · State · Observer

管理端 ▶ CommandBus : StartUseCommand

CommandBus : [AUDIT] → execute()

Service ▶ Vehicle : markInUse() + save

Service ▶ State : startUse() → InUse

Service ▶ Observer : notifyStarted · Email

◀ 200 OK · IN_USE

④ 還車

→ RETURNED

POST /{id}/complete

CommandStateObserver自動違規

管理端 · CommandBus · Service · Vehicle · Observer · ViolationService

管理端 ▶ CommandBus : CompleteCommand

Service ▶ Vehicle : markAvailable + 里程

Service ▶ State : complete() → Returned

Service ▶ Observer : notifyCompleted · Email

opt 逾時 ▶ ViolationService : 建 OVERDUE

◀ 200 OK · RETURNED

OOD 原則對照

PRINCIPLE	應用位置
SRP 單一職責	Service 拆為 Borrowing / Vehicle / Maintenance / User，各有單一修改理由
OCP 開放封閉	Factory 新增角色、Observer 新增管道、Strategy 替換演算法，皆免改舊程式碼
DIP 依賴反轉	Service 依賴 Repository 介面而非 JPA 實作；Strategy 與 Observer 依抽象注入
LoD 迪米特法則	Service 呼叫 request.approve() 委派給 State，不直接操作狀態字串
DRY 不重複	AbstractProtectedService 統一權限守衛，消除各 Service 重複的 if-throw
組合優於繼承	User 持有 Set<Role>、Role 持有 Set<Permission>，角色以組合擴充

測試策略

單元測試

InMemory · 毫秒級

以 InMemory Repository 實作取代資料庫，不依賴 DB 即可驗證 Service 邏輯。

- › BorrowingServiceTest 送審 / 核准 / 拒絕 / 衝突 / 完整流程
- › VehicleServiceTest 建立 / 查詢 / 刪除 / 權限
- › MaintenanceServiceTest 新增 / 到期提醒 / 權限
- › UserServiceTest 密碼雜湊 / 重複 email / 角色權限

整合測試

@WebMvcTest · HTTP

以 MockBean 測試 REST 控制器的完整 HTTP 處理鏈。

- › HTTP 狀態碼與 JSON 序列化
- › JWT 認證守衛驗證
- › 權限拒絕 403 · 未認證 401

| 可獨立測試 Service，正是 **DIP** 帶來的直接收益。

CI/CD 自動部署



只在推送 v* tag 時觸發，不監聽 push to main — 精準控制部署時機、節省 Actions 額度。

未來規劃與結語

後續可擴充方向

Observer 新增 SMS / Line 通知管道（目前 Email+站內收件夾）

Decorator 正式環境啟用 30 分鐘緩衝策略

Chain 新增「每月申請次數上限」驗證器

Command 命令佇列、非同步執行與重試

三個收穫

- 1 10 個 GoF 模式對應真實擴充痛點，其中 4 個用於重構既有設計。
- 2 SOLID · LoD · DRY 等 OOD 原則貫穿分層架構。
- 3 介面與 DIP 讓系統可測試、可替換、可擴充。

王建葦 D1210799 · 陳稚翔 D1249623 demo.jw-albert.dev
· 謝謝聆聽